Lesson 3: Take Your Shot

Unreal Engine (optional advanced track)

VIDEO GAME CREATION

This week: spawning objects at runtime. Your job is to make your player shoot a projectile that flies up the screen and cleans itself up when it leaves - the first piece of your capstone's actual shooting mechanic.

Open your own [YourName]Game project from Lesson 2 - don't start a new one. Everything this week builds on the player object you already have.

Tip: this lesson's Assets folder has a ready-to-use placeholder projectile image (bullet_yellow.png) and a new sound effect (sfx_shoot.wav) - you don't need to find or make your own art today.

Setup

1. Open your [YourName]Game project from Lesson 2.
2. Import bullet_yellow.png (this lesson's Assets folder) into the Content Browser the same way Lesson 2 imported the player circles, then build M_Bullet_Yellow the same way Lesson 2 built M_Player_Orange - a new Material, Shading Model = Unlit, a Texture Sample using bullet_yellow.png wired straight into Emissive Color.
3. Create a new Blueprint class (Actor parent), name it BP_Bullet. Add a Static Mesh Component to it and set its shape to a small basic Shape (e.g. Sphere, built into the Blank template). Assign M_Bullet_Yellow to it in the Static Mesh Component's Material slot - a default Shape's built-in material is Lit, which renders solid black with this project's lights removed, so every new Shape from here on needs its own Unlit Material assigned explicitly.
4. Open BP_Bullet and add a public float variable Speed (default something like 400).
5. In the Content Browser, click Add (+) > Input > Input Action, name it IA_Fire. Leave its Value Type as the default (Digital bool), same as the 4 move actions from Lesson 2.
6. Open IMC_Default (the Input Mapping Context you built in Lesson 2) and click + Add Mapping. Choose IA_Fire, then set its Key to Space Bar. Because IMC_Default was already added to the player in Lesson 2's BP_Player BeginPlay, this new mapping is live immediately - no extra Add Mapping Context step is needed just for adding one more action to a context that's already active.

Level 1 - Minimal (MVP)

Pressing the fire key spawns one projectile that flies upward and disappears when it leaves the screen. This is a complete finish for today - reaching this is enough to move on to next week.

7. In BP_Bullet's Event Graph, add an Event Tick node feeding an "Add Actor World Offset" node. Right-click its Delta Location pin and choose "Split Struct Pin" first (it starts collapsed as one Vector pin, and won't accept a value into just Y until it's split), then wire Speed * Delta Seconds into Y. Add a Branch checking its own Y location against an off-screen threshold, feeding a "Destroy Actor" node on True.
8. Back in BP_Player's own Event Graph (from Lesson 2 - not the Level Blueprint), right-click and search for IA_Fire to add an Enhanced Input Action Event node, and wire its Started pin to a "Spawn Actor from Class" node (Class = BP_Bullet). For Transform, add a "Get Actor Transform" node with its Target left unwired (defaults to Self, same as every other node this course has you leave unwired inside BP_Player), and wire its output into Spawn Actor from Class's Transform pin - that's what spawns the bullet at BP_Player's own current position and facing.
9. Click Compile, then Play and press spacebar to test - a bullet should spawn at BP_Player and fly upward, then disappear.

Alternate Path: C++ (code, optional advanced track)

The same spawn/move/cleanup pattern works in C++ instead of Blueprint nodes.

10. Tools > New C++ Class, choose Actor as the parent class, name it CppBullet, click Create Class.
11. In CppBullet.h, declare a float Speed and override Tick().
12. In CppBullet.cpp's Tick(), call AddActorWorldOffset() along Y, then check GetActorLocation().Y against your off-screen threshold and call Destroy() when past it.
13. In your player Actor's class (the same one from Lesson 2's C++ path), add a UInputAction* FireAction UPROPERTY, bind it in SetupPlayerInputComponent() with EnhancedInputComponent->BindAction(FireAction, ETriggerEvent::Started, this, &Class::Fire) alongside the move bindings, then call GetWorld()->SpawnActor<ACppBullet>() inside that Fire() handler, passing the player's current transform.
14. Compile, assign FireAction to the IA_Fire asset in the Details panel, Play, and test with spacebar.

```
// CppBullet.h
virtual void Tick(float DeltaSeconds) override;
float Speed = 400.f;

// CppBullet.cpp
void ACppBullet::Tick(float DeltaSeconds) {
    Super::Tick(DeltaSeconds);
    AddActorWorldOffset(FVector(0.f, Speed * DeltaSeconds, 0.f));
    if (GetActorLocation().Y > 600.f) Destroy();
}

// Player class: UPROPERTY(EditAnywhere) class UInputAction* FireAction;
// In SetupPlayerInputComponent(), alongside the Lesson 2 move bindings:
EnhancedInput->BindAction(FireAction, ETriggerEvent::Started, this,
&APlayerActor::Fire);

void APlayerActor::Fire() {
    GetWorld()->SpawnActor<ACppBullet>(BulletClass, GetActorLocation(),
FRotator::ZeroRotator);
}
```

This mirrors the Blueprint path's Spawn Actor from Class node and BP_Bullet's own Event Tick - same underlying engine calls, just from code. FireAction reuses the exact same IMC_Default and AddMappingContext() call already set up in Lesson 2, so no new mapping-context wiring is needed here, only the new IA_Fire asset and its BindAction. Level 2/3: Enhanced Input's Started trigger only fires once per press in C++ too, so no extra cooldown logic is required for Level 1/2, and UGameplayStatics::PlaySound2D() plus SetActorRelativeScale3D() cover the Level 3 sound and visual ideas.

Level 2 - Complete (Alpha/Beta)

Firing repeatedly produces multiple independent projectiles on screen at once, each moving and disappearing correctly, and rapid-fire is capped so holding the key down doesn't flood the screen.

- Test firing several times quickly - each press should spawn its own independent BP_Bullet instance, all moving and destroying themselves correctly. Since "Spawn Actor from Class" creates a fully separate Actor each call, this already works with no extra nodes.
- Add a cooldown so holding spacebar doesn't fire every frame: an Enhanced Input Action Event's Started pin only fires once per physical press (not every frame held, since it's event-driven rather than polled), so Level 1 is already immune to this specific bug - add a Retriggerable Delay node after the Started pin anyway if you want a slower, tunable fire rate (e.g. 0.2 seconds), gating the Spawn Actor node behind it.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Sound: add a Play Sound 2D node (using an imported sfx_shoot.wav Sound Wave asset) chained after the Spawn Actor from Class node.
- Visual: give BP_Bullet a brief scale pulse on spawn that eases back to normal size after a fraction of a second - Set Actor Relative Scale3D up (e.g. 1.5, 1.5, 1.5), chain a Delay node, then set it back to (1, 1, 1). BP_Bullet's own sprite (bullet_yellow.png, from Setup) already makes it read clearly as "a shot" rather than a copy of the player Cube, so this idea is a pure size flourish, not a permanent recolor.
- Structure: keep everything about how a bullet behaves - its own movement, its own off-screen cleanup - inside BP_Bullet's own Event Graph, and keep BP_Player limited to just spawning it (Spawn Actor from Class + cooldown + sound). Don't add bullet-specific logic to BP_Player, or vice versa.

If Something's Wrong

- Nothing happens on spacebar: confirm you clicked Compile after editing, that IA_Fire actually has a Space Bar mapping inside IMC_Default (not some other, unmapped context), and that the white execution wire connects the Enhanced Input Action Event's Started pin through to Spawn Actor from Class.
- Bullet spawns but doesn't move or never disappears: confirm BP_Bullet's own Event Tick is wired up inside BP_Bullet itself, not accidentally built in BP_Player.
- Bullet renders solid black instead of yellow: open M_Bullet_Yellow and confirm its Texture Sample is actually wired into Emissive Color, and that bullet_yellow.png is actually assigned as its Texture - the same check as Lesson 2's M_Player_Orange.

- This is the most hardware-demanding engine on our list - if your laptop struggles, it's completely fine to observe this week's demo and build the exercise in a lighter engine instead.

Want More?

dev.epicgames.com/community/learning, Student Resource List > Unreal